

从源码到可执行文件

以 C 语言为例

hello 程序的生命周期是从一个高级 C 语言程序开始的，因为这种形式能够被人读懂。然而，为了在系统上运行 hello.c 程序，每条 C 语句都必须被其他程序转化为一系列的低级机器语言指令。然后这些指令按照一种称为可执行目标程序的格式打好包，并以二进制磁盘文件的形式存放起来。目标程序也称为可执行目标文件。

在 Unix 系统上，从源文件到目标文件的转化是由编译器驱动程序完成的：

```
linux> gcc -o hello hello.c
```

在这里，GCC 编译器驱动程序读取源程序文件 hello.c，并把它翻译成一个可执行目标文件 hello。这个翻译过程可分为四个阶段完成，如图 1-3 所示。执行这四个阶段的程序(预处理器、编译器、汇编器和链接器)一起构成了编译系统(compilation system)。



图 1-3 编译系统

- 预处理阶段。预处理器(cpp)根据以字符#开头的命令，修改原始的 C 程序。比如 hello.c 中第 1 行的 #include <stdio.h> 命令告诉预处理器读取系统头文件 stdio.h 的内容，并把它直接插入程序文本中。结果就得到了另一个 C 程序，通常是以 .i 作为文件扩展名。
- 编译阶段。编译器(cc1)将文本文件 hello.i 翻译成文本文件 hello.s，它包含一个汇编语言程序。该程序包含函数 main 的定义，如下所示：

```
1  main:
2      subq    $8, %rsp
3      movl    $.LC0, %edi
4      call   puts
5      movl    $0, %eax
6      addq    $8, %rsp
7      ret
```

定义中 2~7 行的每条语句都以一种文本格式描述了一条低级机器语言指令。

汇编语言是非常有用的，因为它为不同高级语言的不同编译器提供了通用的输出语

言。例如，C 编译器和 Fortran 编译器产生的输出文件用的都是一样的汇编语言。

- 汇编阶段。接下来，汇编器(as)将 hello.s 翻译成机器语言指令，把这些指令打包成一种叫做可重定位目标程序(relocatable object program)的格式，并将结果保存在目标文件 hello.o 中。hello.o 文件是一个二进制文件，它包含的 17 个字节是函数 main 的指令编码。如果我们在文本编辑器中打开 hello.o 文件，将看到一堆乱码。
- 链接阶段。请注意，hello 程序调用了 printf 函数，它是每个 C 编译器都提供的标准 C 库中的一个函数。printf 函数存在于一个名为 printf.o 的单独的预编译好了的目标文件中，而这个文件必须以某种方式合并到我们的 hello.o 程序中。链接器(ld)就负责处理这种合并。结果就得到 hello 文件，它是一个可执行目标文件(或者简称为可执行文件)，可以被加载到内存中，由系统执行。

国内教材（袁书）中额外描述了计算机的结构和层次等概念化知识

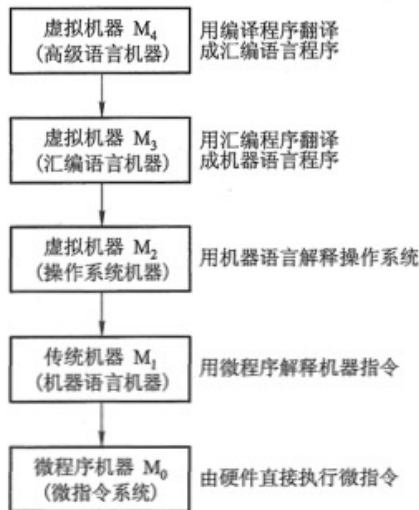


图 1.6 多级层次结构的计算机系统

M0：微程序机器（微指令系统）

现代对应：CPU 的微体系结构（microarchitecture）+ 微码（microcode，部分架构）

- 对 x86 这类复杂指令集：很多“机器指令”内部会被拆成更小的 微操作/微码 来跑（你可以把它理解为“硬件内部的小程序”）。

M1：传统机器（机器语言机器）

现代对应：ISA 指令集层 = x86-64 / ARM64 的机器码

- 你写的任何程序最终都会变成某种 ISA 的机器指令序列（加上数据）。
- CPU “对外”承诺：这些指令语义是稳定的。至于内部 M0 怎么实现，是 CPU 厂商的事。

M1 就是：mov/add/jmp 这种机器级指令真正所在的“抽象机器”。

M2：操作系统机器（OS 虚拟机）

现代对应：OS 内核 + 系统调用接口 + 进程/线程/虚拟内存/文件系统

这一层非常关键：它把 M1 机器“包装”成一个更好用的“虚拟机”，让你觉得自己在用：

- 独立的进程（好像独占 CPU）
- 独立的地址空间（好像独占内存）
- 统一的文件/网络接口（open/read/write/socket...）
- 权限与隔离（用户态/内核态）

M2 你可以理解为：“操作系统提供的那台抽象计算机”（你通过系统调用/ABI 使用它）。

M3：汇编语言机器（汇编→机器码）

现代对应：汇编语言 + 汇编器（assembler）+ 链接器（linker）

- 汇编语言是 M1 的“可读版本”（符号、标签、伪指令）。
- 汇编器把汇编翻译成机器码（M1）。
- 链接器把多个目标文件、库拼起来生成可执行文件/共享库。

M3 是：面向人类的机器指令层，但本质上还是通往 M1 的。

M4：高级语言机器（高级语言→下层）

现代对应：C/C++/Rust/Go/Java/Python 等高级语言的“语义机器”

这里分两大派：

1. AOT 编译型（提前编译）：C/C++/Rust/Go

高级语言 →（编译）→ 汇编/机器码 → OS 跑

2. VM/解释/JIT 型（运行时虚拟机）：Java（JVM）、Python（CPython/PyPy）

高级语言 →（编译成字节码/中间表示）→ VM 解释或 JIT → 本机机器码 → OS 跑

对你关心的 JVM：它其实是在 M4 之上又加了一个“字节码机器”，但完全可以看作：M4 用一个运行时系统把自己落到 M1/M2 上。

相同内容在王道中描述如下

图 1.3 计算机系统的多级层次结构



图 1.3 计算机系统的多级层次结构

命题追踪 ▶ CPU 可以理解和直接执行的指令类型（2024）

第 1 级是微程序机器层，这是一个实在的硬件层，它由机器硬件直接执行微指令。

第 2 级是传统机器语言层，它也是一个实际的机器层，由微程序解释机器指令系统。

第 3 级是操作系统层，它由操作系统程序实现。操作系统程序是由机器指令和广义指令组成的，这些广义指令是为了扩展机器功能而设置的，是由操作系统定义和解释的软件指令，所以这一层也称混合层。

第 4 级是汇编语言层，这一层由汇编程序支持和执行，借此可编写汇编语言源程序。

第 5 级是高级语言层，它是面向用户的，是为方便用户编写应用程序而设置的。该层由各种高级语言编译程序支持和执行。在高级语言层之上，还可以有应用程序层，它由解决实际问题的处理程序组成，如文字处理软件、多媒体处理软件和办公自动化软件等。

没有配备软件的纯硬件系统称裸机。第 3 层～第 5 层称为虚拟机器，简单来说就是软件实现的机器。虚拟机器只对该层的观察者存在，这里的分层和计算机网络的分层类似，对于某层的观察者来说，其只能通过该层的语言来了解和使用计算机，而不必关心下层是如何工作的。

层次之间的关系紧密，下层是上层的基础，上层是下层的扩展。

软件和硬件之间的界面就是指令集体系结构（ISA），ISA 定义了一台计算机可以执行的所有指令的集合，每条指令规定了计算机执行什么操作，以及所处理的操作数存放的地址空间和操作数类型。可以看出，ISA 是指软件能感知到的部分，也称软件可见部分。

指令执行简述

3. 指令执行过程的描述

可执行文件中的代码段是由一条一条机器指令构成的,指令是用 0 和 1 表示的一串 0/1 序列,用来指示 CPU 完成一个特定的原子操作。例如,取数指令从存储单元中取出一个数据送到 CPU 的寄存器中,存数指令将 CPU 寄存器的内容写入一个存储单元,ALU 指令将两个寄存器的内容进行某种算术或逻辑运算后送到一个 CPU 寄存器中,等等。指令的执行过程在第 5 章中详细描述。下面以取数指令(送至运算器的 ACC 中)为例来说明,其信息流程如下:

1) 取指令: $PC \rightarrow MAR \rightarrow M \rightarrow MDR \rightarrow IR$

根据 PC 取指令到 IR。将 PC 的内容送 MAR, MAR 中的内容直接送地址线,同时控制器将读信号送读/写信号线,主存储器根据地址线上的地址和读信号,从指定存储单元读出指令,送到数据线上,MDR 从数据线接收指令信息,并传送到 IR 中。

2) 分析指令: $OP(IR) \rightarrow CU$

指令译码并送出控制信号。控制器根据 IR 中指令的操作码,生成相应的控制信号,送到不同的执行部件。在本例中,IR 中是取数指令,因此读控制信号被送到总线的控制线上。

3) 执行指令: $Ad(IR) \rightarrow MAR \rightarrow M \rightarrow MDR \rightarrow ACC$

取数操作。将 IR 中指令的地址码送 MAR, MAR 中的内容送地址线,同时控制器将读信号送读/写信号线,从主存储器中读出操作数,并通过数据线送至 MDR,再传送到 ACC 中。

每取完一条指令,还需为取下一条指令做准备,计算下一条指令的地址,即 $(PC) + 1 \rightarrow PC$ 。

注意

(PC)指程序计数器 PC 中存放的内容。 $PC \rightarrow MAR$ 应理解为 $(PC) \rightarrow MAR$,即程序计数器中的值经过数据通路送到 MAR,也表示数据通路时括号可省略(因为只是表示数据流经的途径,而不强调数据本身的流动)。但运算时括号不能省略,即 $(PC) + 1 \rightarrow PC$ 不能写为 $PC + 1 \rightarrow PC$ 。当题目中 $(PC) \rightarrow MAR$ 的括号未省略时,考生最好也不要省略。

三种语言的区别

2. 三个级别的语言

命题追踪 ▶ 三种机器语言的特点 (2015)

- 1) 机器语言。也称二进制代码语言,需要编程人员记忆每条指令的二进制编码。机器语言是计算机唯一可以直接识别和执行的语言。
- 2) 汇编语言。汇编语言用英文单词或其缩写代替二进制的指令代码,更容易为人们记忆和理解。使用汇编语言编写的程序,必须经过一个称为汇编程序的系统软件的翻译,将其转换为机器语言程序后,才能在计算机的硬件系统上执行。
- 3) 高级语言。高级语言(如 C、C++、Java 等)可以方便程序设计人员写出解决问题的处理方案和解题过程的程序。通常高级语言需要经过编译程序编译成汇编语言程序,然后经过汇编操作得到机器语言程序,或直接由高级语言程序翻译成机器语言程序。

命题追踪 ▶ 各种翻译程序的概念 (2016)

因为计算机无法直接理解和执行高级语言程序,所以需要将高级语言程序转换为机器语言程序,通常把进行这种转换的系统软件称翻译程序。翻译程序有以下三类:

- 1) 汇编程序(汇编器)。将汇编语言程序翻译成机器语言程序。
- 2) 解释程序(解释器)。将源程序中的语句按执行顺序逐条翻译成机器指令并立即执行。
- 3) 编译程序(编译器)。将高级语言程序翻译成汇编语言或机器语言程序。

3. 软件和硬件的逻辑功能等价性

硬件实现的往往是最基本的算术和逻辑运算功能,而其他功能大多通过软件的扩充得以实现。对某一功能来说,其既可以由硬件实现,又可以由软件实现,从用户的角度来看,它们在功能上是等价的。这一等价性被称为软/硬件逻辑功能的等价性。例如,浮点数运算既可以用专门的浮点运算器硬件实现,又可以通过一段子程序实现,这两种方法在功能上完全等价,不同的只是执行时间的长短而已,显然硬件实现的性能要优于软件实现的性能。